



TRABAJO PRÁCTICO INTEGRADOR

Gestión de Datos de Países en Python: Filtros, Ordenamientos y Estadísticas

Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación

Programación 1

Grupo 17

Integrantes

Mateo Fernandez

Nayla Benitez

Fecha de Entrega

13/06/2026

ÍNDICE

1. Introducción
2. Marco Teórico
 - 2.1 Listas
 - 2.2 Diccionarios
 - 2.3 Funciones
 - 2.4 Estructuras Condicionales
 - 2.5 Estructuras Repetitivas
 - 2.6 Ordenamientos
 - 2.7 Estadísticas Básicas
 - 2.8 Archivos CSV
3. Decisiones Técnicas y Arquitectura
4. Dificultades Encontradas
5. Conclusiones
6. Bibliografía / Webgrafía
7. Links

1. INTRODUCCIÓN

El presente Trabajo Práctico Integrador fue desarrollado en el marco de la asignatura Programación 1 de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional.

El objetivo principal del proyecto consiste en desarrollar una aplicación de consola utilizando Python capaz de gestionar información de países almacenada en un archivo CSV. El sistema permite realizar operaciones de búsqueda, filtrado, ordenamiento y generación de estadísticas, aplicando los conceptos fundamentales estudiados durante la cursada.

A través de este trabajo se busca afianzar el uso de estructuras de datos, funciones, validaciones de entrada, manejo de archivos y técnicas de modularización del código.

2. MARCO TEÓRICO

2.1 Listas

Las listas constituyen una de las estructuras de datos fundamentales de Python, ya que permiten almacenar múltiples elementos dentro de una misma variable manteniendo un orden determinado.

En este proyecto, la lista representa la estructura principal de almacenamiento en memoria. Una vez leídos los registros del archivo CSV, cada país es incorporado a una lista llamada países, que funciona como colección central de datos sobre la cual operan todas las funcionalidades del sistema.

La utilización de listas permitió implementar recorridos secuenciales para mostrar información, realizar búsquedas, aplicar filtros, ejecutar algoritmos de ordenamiento y calcular estadísticas.

Ejemplo

```
países = []
```

A medida que se procesa el archivo CSV, cada nuevo país es agregado mediante el método `append()`, formando una colección dinámica que puede crecer o modificarse durante la ejecución del programa.

2.2 Diccionarios

Los diccionarios son estructuras de datos que almacenan información mediante pares clave-valor. Su principal ventaja consiste en permitir el acceso a los datos a través de nombres descriptivos en lugar de posiciones numéricas.

Dentro del sistema, cada país se representa mediante un diccionario que contiene los atributos necesarios para su identificación y análisis: nombre, población, superficie y continente.

Esta decisión facilita la lectura del código y simplifica operaciones como búsquedas, filtros, actualizaciones y generación de estadísticas, ya que cada dato puede recuperarse utilizando una clave significativa.

Ejemplo

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}
```

La combinación entre listas y diccionarios permitió construir una estructura de datos flexible y adecuada para los requerimientos del proyecto, ya que la información puede recorrerse fácilmente y al mismo tiempo mantenerse organizada.

2.3 Funciones

Las funciones permiten dividir un programa en módulos independientes, agrupando instrucciones que realizan una tarea específica.

Durante el desarrollo del proyecto se adoptó un enfoque modular, donde cada funcionalidad principal fue encapsulada dentro de una función distinta. Esta decisión mejora la organización del código, facilita su mantenimiento y reduce la repetición de instrucciones.

Por ejemplo, existen funciones específicas para cargar los datos desde el archivo CSV, agregar países, actualizar información, realizar búsquedas, aplicar filtros, ordenar registros y generar estadísticas.



Esta metodología también responde a uno de los criterios establecidos por la consigna, que solicita una estructura modular donde cada función posea una responsabilidad claramente definida.

Ejemplo

```
def buscar_pais(nombre):  
    pass
```

Gracias a esta organización, cada módulo puede modificarse o ampliarse sin afectar significativamente el funcionamiento general del sistema.

2.4 Estructuras Condicionales

Las estructuras condicionales permiten que un programa tome decisiones en función de determinadas condiciones. Gracias a ellas es posible ejecutar diferentes acciones según el estado de los datos o las entradas realizadas por el usuario.

Dentro de este proyecto, las estructuras condicionales fueron utilizadas principalmente para validar información ingresada por el usuario, controlar errores y gestionar el flujo lógico de la aplicación.

Por ejemplo, se utilizan para verificar que los nombres de países no estén vacíos, impedir el ingreso de poblaciones o superficies negativas, determinar si un país existe antes de actualizarlo y controlar opciones inválidas dentro del menú principal.

Ejemplo

```
if opcion == 1:  
    agregar_pais()  
  
if poblacion <= 0:  
    print("Error: La población debe ser un número entero positivo.")
```

La incorporación de estas validaciones mejora la robustez del sistema y evita errores que podrían comprometer la integridad de los datos almacenados.

2.5 Estructuras Repetitivas



Las estructuras repetitivas permiten ejecutar un conjunto de instrucciones múltiples veces sin necesidad de escribir el mismo código repetidamente.

En el desarrollo de este sistema se utilizaron principalmente ciclos for y while.

Los ciclos “for” fueron empleados para recorrer la lista de países durante operaciones de búsqueda, filtrado, ordenamiento y generación de estadísticas.

Por otro lado, los ciclos “while” se utilizaron para solicitar nuevamente información al usuario cuando los datos ingresados no cumplían con las validaciones establecidas.

Ejemplo

```
for pais in paises:
    print(pais)
```

Gracias al uso de estructuras repetitivas fue posible procesar dinámicamente cualquier cantidad de registros almacenados en el sistema.

2.6 Ordenamientos

El ordenamiento consiste en reorganizar un conjunto de datos siguiendo un criterio específico, facilitando su visualización y análisis.

En este proyecto se implementaron funcionalidades para ordenar los países por nombre, población y superficie.

Para ello se utilizó el algoritmo Burbuja (Bubble Sort), un método de ordenamiento basado en comparaciones sucesivas entre elementos adyacentes. Cuando dos elementos se encuentran en una posición incorrecta, se intercambian hasta obtener el orden deseado.

Aunque Python dispone de funciones integradas para ordenar colecciones, se optó por implementar el algoritmo manualmente debido a su valor didáctico y a su relación con los contenidos abordados durante la cursada.

Los ordenamientos desarrollados permiten visualizar la información de manera más organizada y facilitan posteriores análisis de los datos.

2.7 Estadísticas Básicas

Las estadísticas básicas permiten transformar un conjunto de datos en información útil para el análisis y la toma de decisiones.



Dentro del sistema se implementaron diversas métricas obtenidas a partir de los países almacenados en memoria.

Entre ellas se encuentran:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Para obtener estos resultados fue necesario recorrer la lista de países utilizando variables acumuladoras, comparadores y un diccionario auxiliar encargado de contabilizar los registros correspondientes a cada continente.

Estas estadísticas permiten obtener una visión general del conjunto de datos almacenados y constituyen una herramienta complementaria a las funciones de búsqueda y filtrado.

2.8 Archivos CSV

Los archivos CSV (Comma Separated Values) son un formato ampliamente utilizado para almacenar información estructurada en forma tabular.

En este proyecto, el archivo CSV cumple la función de fuente de datos persistente, permitiendo almacenar la información de los países incluso después de finalizar la ejecución del programa.

Durante el inicio del sistema, los datos son leídos mediante `csv.DictReader()` y convertidos en una lista de diccionarios para su procesamiento. Posteriormente, cuando se agregan o actualizan países, la información es guardada nuevamente utilizando `csv.DictWriter()`.

La utilización de archivos CSV permitió implementar persistencia de datos de forma sencilla, manteniendo la compatibilidad con herramientas externas como hojas de cálculo o editores de texto.

3. DECISIONES TÉCNICAS Y ARQUITECTURA



Objetivo de la Arquitectura

El sistema fue diseñado siguiendo un enfoque modular basado en funciones independientes. Cada módulo del programa cumple una responsabilidad específica, permitiendo mantener una estructura clara, reutilizable y fácil de mantener.

Esta organización facilita la comprensión del código, reduce la duplicación de lógica y permite realizar modificaciones futuras sin afectar significativamente el funcionamiento general del sistema.

Flujo General del Sistema

1. Lectura del archivo CSV.
2. Carga de los países en memoria.
3. Presentación del menú principal.
4. Selección de una opción por parte del usuario.
5. Ejecución de la funcionalidad correspondiente.
6. Visualización de resultados.
7. Actualización del archivo CSV cuando corresponda.
8. Retorno al menú principal.

Este flujo se mantiene durante toda la ejecución hasta que el usuario decide finalizar el programa.

Persistencia de datos

Una decisión importante fue implementar persistencia automática mediante un archivo CSV.

De esta manera, cualquier modificación realizada por el usuario se almacena inmediatamente y permanece disponible en futuras ejecuciones del programa, evitando pérdidas de información.

Normalización de datos

Durante las primeras pruebas se detectó que diferencias de formato como mayúsculas, minúsculas o espacios adicionales podían afectar búsquedas, filtros y estadísticas.

Para resolver este problema se implementó una función de normalización que unifica los nombres de continentes antes de realizar comparaciones, garantizando resultados consistentes independientemente de cómo se hayan ingresado los datos.

Estrategia de validación

Se priorizó la validación temprana de los datos ingresados por el usuario.

Por este motivo se incorporaron controles para evitar campos vacíos, valores negativos y errores de conversión numérica, reduciendo la posibilidad de inconsistencias dentro del sistema.

Implementación de ordenamientos

Para los procesos de ordenamiento se eligió implementar manualmente el algoritmo Burbuja.

Si bien existen alternativas más eficientes y funciones integradas en Python, esta decisión se tomó con fines educativos, permitiendo aplicar de forma práctica los conceptos vistos durante la cursada y comprender el funcionamiento interno de los algoritmos de ordenamiento.

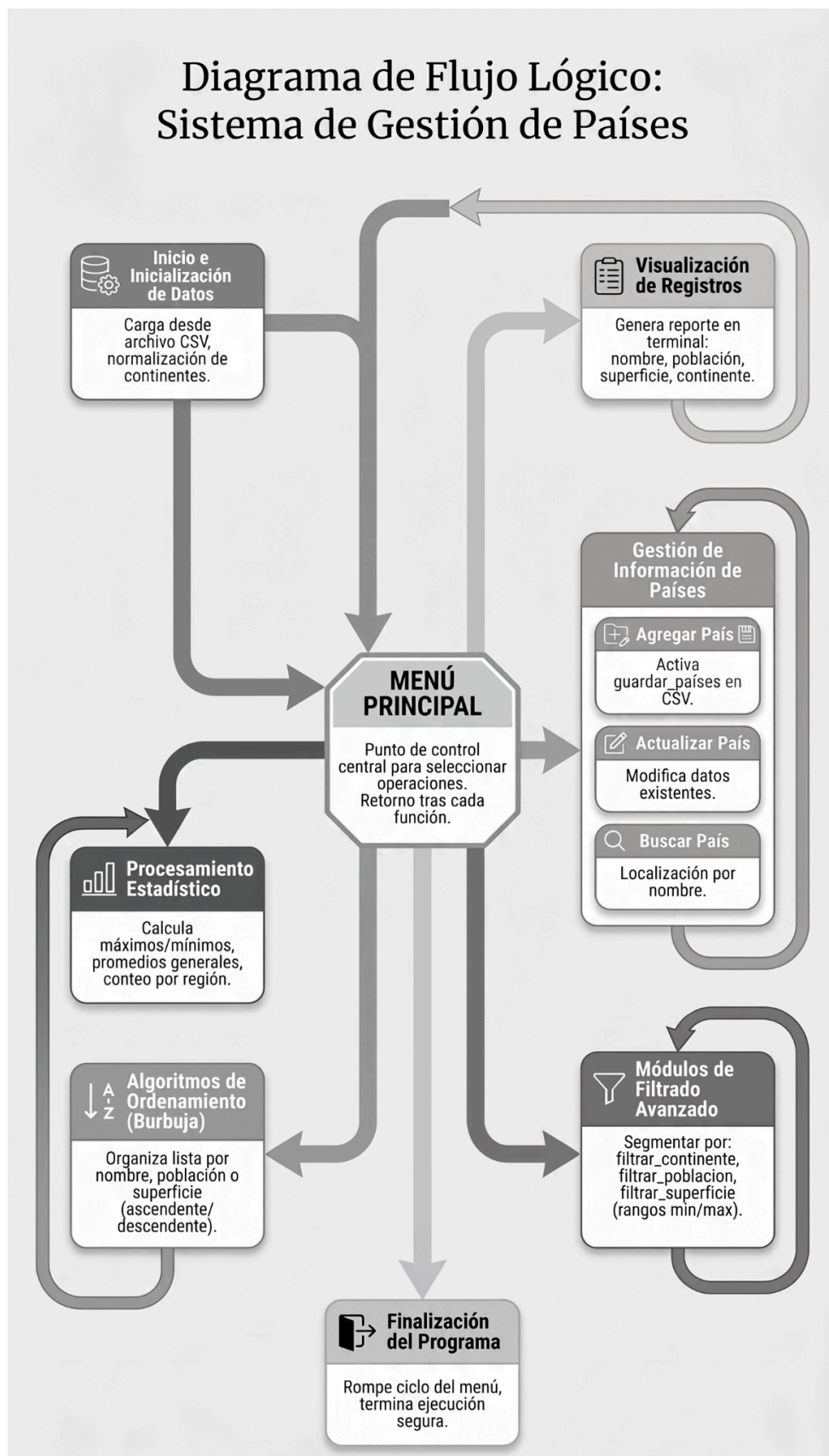
Arquitectura general

La arquitectura final del sistema se basa en tres componentes principales:

- Capa de persistencia: lectura y escritura del archivo CSV.
- Capa de procesamiento: búsquedas, filtros, ordenamientos y estadísticas.
- Capa de interacción: menú principal y comunicación con el usuario.

Esta separación permitió mantener una estructura clara y favoreció la organización general del proyecto.

Diagrama de Flujo Lógico: Sistema de Gestión de Países





Capturas de Ejecución

- Menú Principal:

```
===== TRABAJO PRACTICO INTEGRADOR =====
-----Programacion 1-----

¡Bienvenido al programa de gestión de países!

Alumnos integrantes: Mateo Fernández, Nayla Benitez - Grupo 17

===== MENU =====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país
5. Filtrar por continente
6. Filtrar por población
7. Filtrar por superficie
8. Ordenar por nombre (A-Z)
9. Ordenar por población (de menor a mayor)
10. Ordenar por superficie
11. Mostrar estadísticas
0. Salir

Seleccione una opción: █
```

- Búsqueda por país:

```
===== MENU =====
1. Mostrar países
2. Agregar país
3. Actualizar país
4. Buscar país
5. Filtrar por continente
6. Filtrar por población
7. Filtrar por superficie
8. Ordenar por nombre (A-Z)
9. Ordenar por población (de menor a mayor)
10. Ordenar por superficie
11. Mostrar estadísticas
0. Salir

Seleccione una opción: 4
Ingresa el nombre a buscar: Argentina

===== RESULTADOS DE BÚSQUEDA =====

Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: America
```



- Filtros:

Seleccione una opción: 5
Continente: Europa

===== RESULTADOS POR CONTINENTE =====

Nombre: Alemania | Población: 83149300 | Superficie: 357022 km² | Continente: Europa
Nombre: Francia | Población: 67391582 | Superficie: 551695 km² | Continente: Europa
Nombre: Espana | Población: 47351567 | Superficie: 505990 km² | Continente: Europa
Nombre: Italia | Población: 59554023 | Superficie: 301340 km² | Continente: Europa
Nombre: Reino Unido | Población: 68207114 | Superficie: 243610 km² | Continente: Europa
Nombre: Albania | Población: 2715000 | Superficie: 28748 km² | Continente: Europa

Seleccione una opción: 6
Población mínima: 10000
Población máxima: 5000000

===== RESULTADOS POR POBLACIÓN =====

Nombre: Uruguay | Población: 3473727 | Superficie: 176215 km² | Continente: America
Nombre: Albania | Población: 2715000 | Superficie: 28748 km² | Continente: Europa

Seleccione una opción: 7
Superficie mínima: 700000
Superficie máxima: 400000000

===== RESULTADOS POR SUPERFICIE =====

Nombre: Argentina | Población: 45376763 | Superficie: 2780400 km² | Continente: America
Nombre: Brasil | Población: 213993437 | Superficie: 8515767 km² | Continente: America
Nombre: Chile | Población: 20000000 | Superficie: 800000 km² | Continente: America
Nombre: Estados Unidos | Población: 331893745 | Superficie: 9833517 km² | Continente: America
Nombre: Canada | Población: 38246108 | Superficie: 9984670 km² | Continente: America
Nombre: Mexico | Población: 128932753 | Superficie: 1964375 km² | Continente: America
Nombre: China | Población: 1412600000 | Superficie: 9596961 km² | Continente: Asia
Nombre: India | Población: 1407563842 | Superficie: 3287263 km² | Continente: Asia
Nombre: Australia | Población: 25687041 | Superficie: 7692024 km² | Continente: Oceania
Nombre: Sudafrica | Población: 59308690 | Superficie: 1221037 km² | Continente: Africa
Nombre: Egipto | Población: 109262178 | Superficie: 1002450 km² | Continente: Africa
Nombre: Nigeria | Población: 216746934 | Superficie: 923768 km² | Continente: Africa
Nombre: Peru | Población: 34000000 | Superficie: 1285216 km² | Continente: America

- Estadísticas:

Seleccione una opción: 11

===== ESTADISTICAS =====

Mayor población: China
Menor población: Albania
Promedio población: 190220397
Promedio superficie: 2591876.08 km²

Países por continente:
America : 9
Europa : 6
Asia : 4
Oceania : 2
Africa : 3

4. DIFICULTADES ENCONTRADAS

Durante el desarrollo del proyecto surgieron diversos desafíos técnicos relacionados con la lectura y validación de datos, la implementación de filtros dinámicos y la correcta organización del código en funciones independientes.

Una de las principales dificultades estuvo relacionada con la lectura y escritura del archivo CSV. Fue necesario comprender el funcionamiento del módulo csv de Python para poder transformar correctamente cada fila del archivo en un diccionario utilizable por el programa.

```
try:
    # Guardamos la ruta absoluta del archivo CSV utilizando el módulo os para a
    ruta_script = os.path.dirname(os.path.abspath(__file__))
    # Construimos la ruta completa al archivo CSV utilizando os.path.join para
    ruta_csv = os.path.join(ruta_script, "..", "data", "países.csv")
```

El primer verdadero desafío fue el de poder acceder al archivo .csv desde un directorio distinto al cual estaba alojado el archivo principal. Para ello, necesitamos importar un módulo python, llamado “os”, la cual es una herramienta integrada en las bibliotecas de python para mejorar la gestión de archivos y directorios, además de importar el módulo “csv”, para efectivamente poder leer y escribir los datos necesarios.

Gracias a estos módulos, pudimos guardar en una variable llamada “ruta_script” la obtención de la ruta absoluta (dirección completa y exacta de un archivo o directorio) del archivo python que se está ejecutando, extrayendo el directorio que lo contiene ; Posteriormente, mediante “os.path.join()” se construye la ruta completa hacia el archivo “países.csv” de manera dinámica y compatible con distintos sistemas operativos.

Es como indicarle, que partiendo desde la carpeta en la que está el script, le hacemos subir un nivel (..) , entrar a la carpeta “data” y acceder finalmente al archivo “países.csv”.

También fue necesario implementar validaciones que evitaran errores provocados por entradas inválidas ingresadas por el usuario.

Por ejemplo : En el marco del desarrollo de la lógica del sistema, y del armado de cada módulo, se notaban reiterados fallos en la validación de datos numéricos ingresados por parte del usuario, al momento de decidir o bien las opciones de menú, los rangos poblacionales o de superficie.



El sistema arrojaba errores, en caso el usuario cometa el error de ingresar espacios vacíos antes del / de los dígito/s a ingresar en el sistema.

Este tipo de errores, pudieron resolverse incluyendo en primer lugar, métodos de validación de datos como `.strip()` al final de las sentencias input que debían de recibir los ingresos numerales. Gracias a este método, logramos hacer que el sistema reconozca los números ingresados - en caso de cumplir con las demás condiciones de validez - inclusive cuando se vean precedidas por espacios vacíos.

```
opcion = input(
    "\nSeleccione una opción: "
).strip()
```

```
opcion = input(
    "1-Ascendente | 2-Descendente: "
).strip()
```

Así mismo, este tipo de validaciones, tenían que venir acompañadas de condiciones claras como las de no permitir el guardado de números negativos. Estas sentencias condicionales se incluyeron luego de bloques try/except, en su mayoría, en el marco de un bucle while que pueda volver a generar el mismo flujo de solicitudes al usuario hasta que no ingrese datos validables :

```
while True:
    try:
        maximo = int(input("Superficie máxima: "))
    except ValueError:
        print("Error: La superficie debe ingresarse con un número entero válido.")
        continue
    if maximo < 0 : |
        print("Error: La superficie maxima tiene que ingresarse con un entero positivo.")
        continue
    break
```

También surgieron dificultades al implementar los filtros y búsquedas. Fue necesario contemplar diferencias entre mayúsculas y minúsculas para que las búsquedas resultaran más flexibles y amigables para el usuario. Para resolver este problema se utilizaron métodos como `lower()`, `casefold()` y funciones de normalización.

```
for pais in paises:
    if pais["continente"].casefold() == continente:
```

```
# facilitar la búsqueda.
if pais["nombre"].lower() == nombre.lower():
    while True:
```

En el apartado de estadísticas, uno de los desafíos fue calcular correctamente los valores promedio y determinar los países con mayor y menor población utilizando recorridos sobre la lista de países.

Asimismo, se implementó un diccionario auxiliar para contabilizar automáticamente la cantidad de países pertenecientes a cada continente.

```
# o menor respectivamente
mayor = paises[0]
menor = paises[0]

# Variables acumuladoras
# total de países para
suma_poblacion = 0
suma_superficie = 0

# cada vez que se encuentra un continente
continentes = {}
```

```
for pais in paises:
    if pais["poblacion"] > mayor["poblacion"]:
        mayor = pais

    if pais["poblacion"] < menor["poblacion"]:
        menor = pais

    suma_poblacion += pais["poblacion"]
    suma_superficie += pais["superficie"]

    continente = normalizar_continente(pais["continente"])

    if continente in continentes:
        continentes[continente] += 1
    else:
        continentes[continente] = 1
```

Notamos como utilizando un bucle for, que itere sobre los valores del diccionario principal (aquel con los datos originales del archivo .csv), empezamos a comparar el valor clave de la población contra los valores iniciales de las variables “mayor”/”menor”, asignando el nuevo dato correspondiente a la variable.

Lo mismo sucede con las variables acumuladoras poblacionales y de superficies, que extraen desde los datos guardados en el .csv cada los datos respectivos para realizar sumatorias, país tras país.

Notar que, para normalizar los nombres de los continentes ante el ingreso de este dato por parte del usuario, y la correcta interpretación del sistema, creamos una función auxiliar que pueda tomar como argumento a ese dato para realizar el siguiente proceso :

```
# que pueda dificultar la comparación a 1
def normalizar_continente(continente):
    """Normaliza el nombre del continente
    return continente.strip().title()
```

El argumento (continente), toma el nombre del continente del país sobre el que se itera, normalizando su formato para que su utilización - por ejemplo - dentro de la función “mostrar_estadisticas(países)”, lo cual es clave para que pueda realizar las comparaciones entre datos, y asimilar correctamente aquellos que correspondan a la misma unidad de información a pesar del formato ingresado (minúsculas, mayúsculas, espacios etc)

Todas estas dificultades nos permitieron profundizar la comprensión en conceptos fundamentales de programación y mejorar en definitiva nuestras habilidades de resolución de problemas, al momento de estructurar un código funcional, eficiente y modular.

5. CONCLUSIONES

La realización de este Trabajo Práctico Integrador nos permitió aplicar de manera práctica los principales conceptos abordados durante la cursada de Programación 1.

A través del desarrollo del sistema se trabajó con estructuras de datos fundamentales como listas y diccionarios, funciones, estructuras condicionales y repetitivas, manejo de excepciones y sentencias “with open”, para el procesamiento de archivos CSV. su lectura, y escritura de datos. La integración de estos conceptos dentro de una aplicación funcional como la propuesta, permitió comprender mejor cómo se relacionan entre sí para resolver problemas reales.

Desde el punto de vista técnico, el proyecto nos permitió adquirir experiencia en el diseño de programas modulares, donde cada función cumple con una responsabilidad *específica*. Esta metodología favorece la reutilización del código, ya que hace a la simplificación de las tareas de mantenimiento y mejora la comprensión general del sistema, aportando a la interoperabilidad de los miembros de un equipo sobre un proyecto, como bien podría ser el caso de nuestro programa.

Asimismo, la utilización de GitHub como herramienta de gestión y control de versiones nos permitió reforzar “buenas prácticas” básicas de trabajo colaborativo utilizadas en entornos profesionales de desarrollo de software.

Como resultado final se obtuvo una aplicación capaz de administrar información de países mediante operaciones de carga, actualización, búsqueda, filtrado, ordenamiento y generación de estadísticas, cumpliendo satisfactoriamente con todos los requerimientos establecidos por las consignas del trabajo.

En términos generales, la experiencia resultó altamente enriquecedora y nos llevó a consolidar conocimientos fundamentales que sin duda, nos servirán de base para asignaturas posteriores de programación y desarrollo de software más avanzadas.

6. BIBLIOGRAFÍA / WEBGRAFÍA

Downey, Allen B. (2015). Pensar en Python: Aprende a pensar como un informático. Green Tea Press. Traducción al español.

Python Software Foundation. Python Documentation.
<https://docs.python.org/3/>

W3Schools. Python Tutorial.
<https://www.w3schools.com/python/>

Real Python.
<https://realpython.com/>

Documentación Oficial de Git.
<https://git-scm.com/doc>

Documentación Oficial de GitHub.
<https://docs.github.com/>

Material de cátedra de Programación 1 - UTN.

7. LINKS

Repositorio GitHub:

<https://github.com/MateoFernandez-GH/TPI-Programacion1-Gestion-Paises.git>

Video Demostración:

 Presentacion TPI Programacion 1 - Grupo 17.mp4